AppStore WPF

Полная документация

Теория WPF и Архитектура проекта

Версия документа:	1.0
Дата:	29 марта 2026
Автор:	Perplexity Computer
Платформа:	.NET 8.0 / WPF
Язык:	C# 12

Содержание

ЧАСТЬ 1: ТЕОРИЯ WPF

1.1 Что такое WPF
1.2 XAML — язык разметки
1.3 Система привязки данных (Data Binding)
1.4 Конвертеры значений
1.5 Команды (ICommand) и паттерн Command
1.6 Стили (Style) и шаблоны (ControlTemplate)
1.7 Триггеры
1.8 Анимации в WPF
1.9 Ресурсы и словари ресурсов
1.10 Паттерн MVVM
1.11 Паттерн Repository
1.12 Сервисный слой (Service Layer)
1.13 Entity Framework Core + PostgreSQL
1.14 Хеширование паролей (SHA256)
1.15 Сериализация JSON
1.16 Кастомный курсор

ЧАСТЬ 2: АРХИТЕКТУРА ПРОЕКТА

2.1 Структура файлов
2.2 Модели данных
2.3 Слой данных (Data Layer)
2.4 Сервисный слой
2.5 ViewModels
2.6 Команды
2.7 Конвертеры (все 17)
2.8 Запуск приложения (App.xaml.cs)
2.9 Тёмная/Светлая тема
2.10 Локализация
2.11 NuGet-зависимости

ЧАСТЬ 1

Теория WPF

1.1 Что такое WPF

Windows Presentation Foundation (WPF) — это фреймворк для создания настольных приложений под Windows, входящий в состав .NET. WPF был представлен компанией Microsoft в 2006 году как замена устаревшей технологии Windows Forms (WinForms) и принёс принципиально новый подход к построению пользовательских интерфейсов.

WPF vs WinForms

Характеристика	WinForms	WPF
Рендеринг	GDI+ (программный)	DirectX (аппаратный)
Разметка UI	Код C# / дизайнер	XAML (декларативный)
Масштабируемость	Привязан к пикселям	Независимость от разрешения
Data Binding	Ограниченный	Полноценный, мощный
Стилизация	Ограничена	Стили, шаблоны, темы
Анимации	Сторонние библиотеки	Встроенная система анимаций
Паттерн	MVP / Code-behind	MVVM
Графика	2D только	2D + 3D

DirectX-рендеринг

В отличие от WinForms, который использует GDI+ для отрисовки элементов управления, WPF использует DirectX. Это означает, что рендеринг выполняется видеокартой (GPU), что обеспечивает:

- Плавные анимации без нагрузки на процессор
- Поддержку прозрачности, градиентов, трансформаций
- Аппаратное ускорение отрисовки
- Рендеринг векторной графики без потери качества

Независимость от разрешения

WPF использует аппаратно-независимые единицы (device-independent pixels, DIP). Один DIP равен 1/96 дюйма. При изменении DPI системы интерфейс автоматически масштабируется без размытия или искажений. Это критически важно для современных мониторов с высоким разрешением (4K, Retina).

Декларативный UI на XAML

XAML (eXtensible Application Markup Language) — XML-подобный язык разметки, позволяющий описывать UI декларативно. Каждый XAML-элемент соответствует .NET классу:

```
<Window x:Class="AppStore.Views.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="AppStore" Height="700" Width="1100"
        WindowStartupLocation="CenterScreen"
        Style="{DynamicResource AppWindowStyle}">
    <Grid>
        <!-- Основной контент приложения -->
    </Grid>
</Window>
```

1.2 XAML — язык разметки

XAML (eXtensible Application Markup Language) — это декларативный язык разметки, основанный на XML. Он позволяет описывать структуру пользовательского интерфейса отдельно от логики приложения, что является фундаментальным принципом разделения ответственности в WPF.

XML-пространства имён (xmlns)

Каждый XAML-файл начинается с объявления пространств имён, которые определяют, какие классы и элементы доступны:

```
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    <!-- Основное: WPF-элементы (Window, Grid, Button, TextBlock и др.) -->

xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    <!-- XAML-инструменты: x:Class, x:Key, x:Name, x:Type -->

xmlns:local="clr-namespace:AppStore"
    <!-- Локальные классы проекта -->

xmlns:converters="clr-namespace:AppStore.Converters"
    <!-- Классы конвертеров значений -->
```

В проекте AppStore файл App.xaml объявляет все необходимые пространства имён для конвертеров и других пользовательских типов:

```
<Application x:Class="AppStore.App"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:converters="clr-namespace:AppStore.Converters">
    <Application.Resources>
        <ResourceDictionary>
            <ResourceDictionary.MergedDictionaries>
                <ResourceDictionary Source="Resources/Styles/AppStyles.xaml"/>
                <ResourceDictionary Source="Resources/Localization/ru.xaml"/>
            </ResourceDictionary.MergedDictionaries>
            <!-- Регистрация конвертеров -->
            <converters:BoolToVisibilityConverter x:Key="BoolToVis"/>
            <converters:PriceConverter x:Key="PriceConv"/>
            <!-- ... -->
        </ResourceDictionary>
    </Application.Resources>
</Application>
```

Типы элементов XAML

ContentControl — элементы с одним содержимым

ContentControl — базовый класс для элементов, которые могут содержать один дочерний элемент. К ним относятся: Window, Button, Label, CheckBox, RadioButton, Border, ScrollViewer.

```
<Button Content="Установить"
        Command="{Binding DownloadCommand}"
        CommandParameter="{Binding SelectedApp}"/>
```

ItemsControl — элементы с коллекцией

ItemsControl содержит коллекцию элементов. К ним относятся: ListBox, ListView, ComboBox, ItemsControl. В проекте ItemsControl используется для отображения списка приложений:

```
<ItemsControl ItemsSource="{Binding Apps}">
    <ItemsControl.ItemTemplate>
        <DataTemplate>
            <local:AppCardView/>
        </DataTemplate>
    </ItemsControl.ItemTemplate>
    <ItemsControl.ItemsPanel>
        <ItemsPanelTemplate>
            <WrapPanel/>
        </ItemsPanelTemplate>
    </ItemsControl.ItemsPanel>
</ItemsControl>
```

Panel — контейнеры компоновки

Панели определяют расположение дочерних элементов:

Панель	Описание	Использование в проекте
Grid	Сетка строк и столбцов	Основная разметка MainWindow
StackPanel	Стек по вертикали/горизонтали	Навигация, списки кнопок
WrapPanel	Перенос по строкам	Сетка карточек приложений
DockPanel	Прикрепление к сторонам	Верхняя панель навигации
Canvas	Абсолютное позиционирование	Не используется

Свойства в XAML

Обычные свойства

Устанавливаются как XML-атрибуты или элементы:

```
<!-- Как атрибут -->
<TextBlock Text="AppStore" FontSize="18" FontWeight="Bold"/>

<!-- Как элемент (для сложных значений) -->
<TextBlock>
    <TextBlock.Text>AppStore</TextBlock.Text>
</TextBlock>
```

Прикреплённые свойства (Attached Properties)

Свойства, определённые в одном классе, но применяемые к другому. Формат: ИмяКласса.ИмяСвойства:

```
<Grid>
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="*"/>
    </Grid.RowDefinitions>
    <TextBlock Grid.Row="0" Text="Заголовок"/>
    <ScrollViewer Grid.Row="1">
```

```
        <!-- Контент -->
    </ScrollViewer>
</Grid>
```

Свойства зависимости (DependencyProperty)

DependencyProperty — расширенная система свойств WPF, поддерживающая привязку данных, анимации, стили, наследование значений и значения по умолчанию. Каждое свойство WPF-элемента, доступное в XAML, является DependencyProperty.

```
// Пример определения DependencyProperty
public static readonly DependencyProperty IsActiveProperty =
    DependencyProperty.Register(
        "IsActive",                      // Имя свойства
        typeof(bool),                    // Тип значения
        typeof(CustomControl),           // Тип-владелец
        new PropertyMetadata(false));    // Метаданные по умолчанию

public bool IsActive
{
    get { return (bool)GetValue(IsActiveProperty); }
    set { SetValue(IsActiveProperty, value); }
}
```

1.3 Система привязки данных (Data Binding)

Data Binding (привязка данных) — механизм автоматической синхронизации данных между UI-элементами и источником данных. Это ключевая технология WPF, обеспечивающая работу паттерна MVVM.

Расширение разметки Binding

Привязка задаётся через расширение разметки {Binding}:

```
<!-- Привязка к свойству DataContext -->
<TextBlock Text="{Binding AppName}"/>

<!-- Привязка к элементу по имени -->
<TextBlock Text="{Binding Text, ElementName=searchBox}"/>

<!-- Привязка к родительскому элементу -->
<TextBlock Text="{Binding DataContext.Title,
    RelativeSource={RelativeSource AncestorType=Window}}"/>
```

Режимы привязки (Binding Mode)

Режим	Направление	Когда используется
OneWay	Источник → Цель	Отображение данных (по умолчанию для TextBlock)
TwoWay	Источник ↔ Цель	Редактирование (TextBox, CheckBox)
OneTime	Источник → Цель (однократно)	Статические данные
OneWayToSource	Цель → Источник	Обратная запись

В проекте AppStore режим TwoWay используется в полях редактирования профиля:

```
<TextBox Text="{Binding FirstName, Mode=TwoWay,
    UpdateSourceTrigger=PropertyChanged}"/>
```

DataContext — контекст данных

DataContext — свойство FrameworkElement, определяющее источник данных по умолчанию для всех привязок элемента и его потомков. DataContext наследуется по визуальному дереву: если у дочернего элемента не установлен свой DataContext, он использует DataContext родителя.

```
// Установка DataContext в code-behind (MainWindow.xaml.cs)
public MainWindow(MainViewModel viewModel)
{
    InitializeComponent();
    DataContext = viewModel;  // Все привязки внутри окна будут
                              // использовать MainViewModel
}
```

INotifyPropertyChanged

Интерфейс INotifyPropertyChanged уведомляет UI об изменении свойства. Без реализации этого интерфейса привязка не узнает об обновлении данных.

```
// ViewModelBase.cs — базовый класс всех ViewModel в проекте
public class ViewModelBase : INotifyPropertyChanged
{
    public event PropertyChangedEventHandler? PropertyChanged;

    protected void OnPropertyChanged(
        [CallerMemberName] string? prop = null)
    {
        PropertyChanged?.Invoke(this,
            new PropertyChangedEventArgs(prop));
    }

    protected bool SetField<T>(ref T field, T value,
        [CallerMemberName] string? prop = null)
    {
        if (EqualityComparer<T>.Default.Equals(field, value))
            return false;
        field = value;
        OnPropertyChanged(prop);
        return true;
    }
}
```

Метод SetField<T> — универсальный сеттер, который сравнивает старое и новое значение, обновляет поле и вызывает событие PropertyChanged только при реальном изменении. Атрибут [CallerMemberName] автоматически подставляет имя вызывающего свойства.

Примеры привязок из проекта

FeaturedBannerView — привязка данных баннера

```
<TextBlock Text="{Binding Name}" FontSize="22"
    FontWeight="Bold" Foreground="White"/>
<TextBlock Text="{Binding Description}" FontSize="13"
    Foreground="#CCFFFFFF" TextWrapping="Wrap"/>
<TextBlock Text="{Binding Price,
    Converter={StaticResource PriceConv}}"
    FontSize="14" Foreground="White"/>
```

MainWindow — привязки навигации и фильтрации

```
<TextBox Text="{Binding SearchQuery, Mode=TwoWay,
    UpdateSourceTrigger=PropertyChanged}"
    Style="{DynamicResource SearchBoxStyle}"/>

<ItemsControl ItemsSource="{Binding Apps}">
    <!-- Отображение отфильтрованных приложений -->
</ItemsControl>

<TextBlock Text="{Binding SelectedApp.Name}"
    FontSize="24" FontWeight="Bold"/>
```

1.4 Конвертеры значений (IValueConverter)

Конвертеры значений позволяют преобразовывать данные при привязке между источником (ViewModel) и целью (элемент UI). Они реализуют интерфейс IValueConverter или IMultiValueConverter.

Интерфейс IValueConverter

```
public interface IValueConverter
{
    // Преобразование из источника в UI
    object Convert(object value, Type targetType,
        object parameter, CultureInfo culture);

    // Обратное преобразование (UI → источник)
    object ConvertBack(object value, Type targetType,
        object parameter, CultureInfo culture);
}
```

Интерфейс IMultiValueConverter

IMultiValueConverter принимает массив значений из MultiBinding. Используется когда конвертеру нужно несколько входных значений одновременно:

```
public interface IMultiValueConverter
{
    object Convert(object[] values, Type targetType,
        object parameter, CultureInfo culture);

    object[] ConvertBack(object value, Type[] targetTypes,
        object parameter, CultureInfo culture);
}
```

Регистрация конвертеров в App.xaml

Все конвертеры регистрируются как StaticResource в ресурсах приложения:

```
<Application.Resources>
    <ResourceDictionary>
        <converters:BoolToVisibilityConverter x:Key="BoolToVis"/>
        <converters:InverseBoolToVisibilityConverter
            x:Key="InvBoolToVis"/>
        <converters:NullToVisibilityConverter x:Key="NullToVis"/>
        <converters:StringNotEmptyToVisibilityConverter
            x:Key="StrNotEmptyToVis"/>
        <converters:PriceConverter x:Key="PriceConv"/>
        <converters:AppGetLabelConverter x:Key="AppGetLabelConv"/>
        <converters:DownloadButtonTextConverter
            x:Key="DownloadBtnTextConv"/>
        <converters:DownloadCountConverter x:Key="DownloadCountConv"/>
        <converters:RatingToStarsConverter x:Key="RatingToStarsConv"/>
        <converters:SizeConverter x:Key="SizeConv"/>
        <converters:SizeNumberConverter x:Key="SizeNumConv"/>
        <converters:SizeUnitConverter x:Key="SizeUnitConv"/>
        <converters:StringToColorBrushConverter
            x:Key="StrToColorConv"/>
        <converters:DiscountVisibilityConverter
```

```
            x:Key="DiscountVisConv"/>
        <converters:DiscountWithPriceVisibilityConverter
            x:Key="DiscountWithPriceVisConv"/>
        <converters:StockToColorConverter x:Key="StockToColorConv"/>
        <converters:RoleNameConverter x:Key="RoleNameConv"/>
    </ResourceDictionary>
</Application.Resources>
```

Все 17 конвертеров проекта

#	Конвертер	Вход → Выход	Описание
1	BoolToVisibilityConverter	bool → Visibility	true=Visible, false=Collapsed
2	InverseBoolToVisibilityConverter	bool → Visibility	true=Collapsed, false=Visible
3	NullToVisibilityConverter	object → Visibility	null=Collapsed, not null=Visible
4	StringNotEmptyToVisibility	string → Visibility	Пустая строка=Collapsed
5	PriceConverter	decimal → string	0=Бесплатно/Free, иначе цена+валюта
6	AppGetLabelConverter	bool → string	Скачано: Открыть/Установить
7	DownloadButtonTextConverter	bool → string	Текст кнопки скачивания
8	DownloadCountConverter	int → string	1500→1.5K, 1000000→1M
9	RatingToStarsConverter	double → string	4.5 → ★★★★½☆
10	SizeConverter	double → string	Полный размер: 150 MB
11	SizeNumberConverter	double → string	Только число: 150
12	SizeUnitConverter	double → string	Только единица: MB/GB
13	StringToColorBrushConverter	#hex → Brush	#FF0000→SolidColorBrush
14	DiscountVisibilityConverter	int → Visibility	discount>0: Visible
15	DiscountWithPriceVisibility	IMultiValueConverter	Скидка>0 И цена>0
16	StockToColorConverter	int → Brush	Цвет по остатку на складе
17	RoleNameConverter	UserRole → string	Admin→Администратор

Пример: PriceConverter (с локализацией валюты)

```
public class PriceConverter : IValueConverter
{
    public object Convert(object value, Type targetType,
        object parameter, CultureInfo culture)
    {
        if (value is decimal price)
        {
            if (price == 0)
            {
                // Проверяем текущий язык из ресурсов
                var lang = Application.Current.Resources
                    .MergedDictionaries[1].Source.ToString();
                return lang.Contains("ru")
                    ? "Бесплатно" : "Free";
            }
            var symbol = lang.Contains("ru") ? "₽" : "$";
            return $"{price:F2} {symbol}";
        }
        return "N/A";
    }

    public object ConvertBack(object value, Type targetType,
        object parameter, CultureInfo culture)
        => throw new NotImplementedException();
}
```

Пример: RatingToStarsConverter

```
public class RatingToStarsConverter : IValueConverter
{
    public object Convert(object value, Type targetType,
        object parameter, CultureInfo culture)
    {
        if (value is double rating)
        {
            int fullStars = (int)Math.Floor(rating);
            bool halfStar = (rating - fullStars) >= 0.5;
            int emptyStars = 5 - fullStars - (halfStar ? 1 : 0);

            return new string('★', fullStars)
                 + (halfStar ? "½" : "")
                 + new string('☆', emptyStars);
        }
        return "☆☆☆☆☆";
    }

    public object ConvertBack(object value, Type targetType,
        object parameter, CultureInfo culture)
        => throw new NotImplementedException();
}
```

Пример: DownloadCountConverter (K/M форматирование)

```
public class DownloadCountConverter : IValueConverter
{
    public object Convert(object value, Type targetType,
        object parameter, CultureInfo culture)
    {
        if (value is int count)
        {
            if (count >= 1_000_000)
                return $"{count / 1_000_000.0:F1}M";
            if (count >= 1_000)
                return $"{count / 1_000.0:F1}K";
            return count.ToString();
        }
        return "0";
    }

    public object ConvertBack(object value, Type targetType,
        object parameter, CultureInfo culture)
        => throw new NotImplementedException();
}
```

Пример: DiscountWithPriceVisibilityConverter (IMultiValueConverter)

```
public class DiscountWithPriceVisibilityConverter
    : IMultiValueConverter
{
    public object Convert(object[] values, Type targetType,
        object parameter, CultureInfo culture)
    {
        if (values.Length == 2
            && values[0] is int discount
            && values[1] is decimal price)
        {
            return (discount > 0 && price > 0)
                ? Visibility.Visible
                : Visibility.Collapsed;
        }
        return Visibility.Collapsed;
    }

    public object[] ConvertBack(object value,
        Type[] targetTypes, object parameter,
        CultureInfo culture)
        => throw new NotImplementedException();
}

<!-- Использование в XAML -->
<TextBlock>
    <TextBlock.Visibility>
        <MultiBinding Converter=
            "{StaticResource DiscountWithPriceVisConv}">
            <Binding Path="Discount"/>
            <Binding Path="Price"/>
        </MultiBinding>
    </TextBlock.Visibility>
</TextBlock>
```

1.5 Команды (ICommand) и паттерн Command

Паттерн Command в WPF инкапсулирует действие в объект, позволяя привязывать его к UI-элементам через Data Binding. Это ключевой механизм MVVM для обработки пользовательских действий без code-behind.

Интерфейс ICommand

```
public interface ICommand
{
    // Выполнить действие
    void Execute(object? parameter);

    // Можно ли выполнить (включена/отключена кнопка)
    bool CanExecute(object? parameter);

    // Событие изменения доступности
    event EventHandler? CanExecuteChanged;
}
```

RelayCommand — реализация из проекта

RelayCommand — обёртка над делегатами Action/Func, позволяющая создавать команды без отдельного класса для каждой:

```
public class RelayCommand : ICommand
{
    private readonly Action<object?> _execute;
    private readonly Func<object?, bool>? _canExecute;

    public RelayCommand(Action<object?> execute,
        Func<object?, bool>? canExecute = null)
    {
        _execute = execute
            ?? throw new ArgumentNullException(nameof(execute));
        _canExecute = canExecute;
    }

    public event EventHandler? CanExecuteChanged
    {
        add => CommandManager.RequerySuggested += value;
        remove => CommandManager.RequerySuggested -= value;
    }

    public bool CanExecute(object? parameter)
        => _canExecute?.Invoke(parameter) ?? true;

    public void Execute(object? parameter)
        => _execute(parameter);
}
```

RelayCommand<T> — типизированная версия

```
public class RelayCommand<T> : ICommand
{
    private readonly Action<T?> _execute;
    private readonly Func<T?, bool>? _canExecute;

    public RelayCommand(Action<T?> execute,
        Func<T?, bool>? canExecute = null)
    {
        _execute = execute
            ?? throw new ArgumentNullException(nameof(execute));
        _canExecute = canExecute;
    }

    public event EventHandler? CanExecuteChanged
    {
        add => CommandManager.RequerySuggested += value;
        remove => CommandManager.RequerySuggested -= value;
    }

    public bool CanExecute(object? parameter)
    {
        if (parameter is T typedParam)
            return _canExecute?.Invoke(typedParam) ?? true;
        return _canExecute?.Invoke(default) ?? true;
    }

    public void Execute(object? parameter)
    {
        if (parameter is T typedParam)
            _execute(typedParam);
        else
            _execute(default);
    }
}
```

CommandManager.RequerySuggested

CommandManager.RequerySuggested — статическое событие WPF, которое срабатывает при любом взаимодействии пользователя (клик, нажатие клавиши, смена фокуса). Подписка на это событие через CanExecuteChanged позволяет автоматически обновлять состояние кнопок (включена/отключена) без ручного вызова.

Специализированные команды проекта

DownloadAppCommand

```
public class DownloadAppCommand : ICommand
{
    private readonly IAppRepository _repo;
    private readonly Action _onCompleted;

    public DownloadAppCommand(IAppRepository repo,
        Action onCompleted)
    {
        _repo = repo;
        _onCompleted = onCompleted;
    }

    public event EventHandler? CanExecuteChanged
    {
        add => CommandManager.RequerySuggested += value;
        remove => CommandManager.RequerySuggested -= value;
    }

    public bool CanExecute(object? parameter) => true;

    public void Execute(object? parameter)
    {
        if (parameter is AppItem app)
        {
            if (app.IsDownloaded)
                _repo.Uninstall(app.Id);
            else
                _repo.Download(app.Id);
            _onCompleted?.Invoke();
        }
    }
}
```

DeleteAppCommand

```
public class DeleteAppCommand : ICommand
{
    private readonly IAppRepository _repo;
    private readonly Action<Guid> _onDeleted;

    public DeleteAppCommand(IAppRepository repo,
        Action<Guid> onDeleted)
    {
        _repo = repo;
        _onDeleted = onDeleted;
    }

    public event EventHandler? CanExecuteChanged
    {
        add => CommandManager.RequerySuggested += value;
        remove => CommandManager.RequerySuggested -= value;
    }

    public bool CanExecute(object? parameter) => true;

    public void Execute(object? parameter)
    {
        if (parameter is Guid appId)
        {
            _repo.Delete(appId);
            _onDeleted?.Invoke(appId);
        }
    }
}
```

Привязка команд в XAML

```
<!-- Простая привязка команды -->
<Button Content="Поиск" Command="{Binding SearchCommand}"/>

<!-- Команда с параметром -->
<Button Content="{Binding SelectedApp.IsDownloaded,
        Converter={StaticResource DownloadBtnTextConv}}"
    Command="{Binding DownloadCommand}"
    CommandParameter="{Binding SelectedApp}"/>

<!-- Команда удаления -->
<Button Content="Удалить"
    Command="{Binding DeleteCommand}"
    CommandParameter="{Binding SelectedApp.Id}"/>
```

1.6 Стили (Style) и шаблоны (ControlTemplate)

Стили в WPF позволяют определить набор значений свойств и применить их к множеству элементов. Шаблоны (ControlTemplate) позволяют полностью переопределить визуальную структуру элемента управления.

Неявные и явные стили

Неявный стиль (без x:Key)

Применяется автоматически ко всем элементам указанного типа:

```
<!-- Применится ко ВСЕМ TextBlock в области действия -->
<Style TargetType="TextBlock">
    <Setter Property="FontFamily" Value="Segoe UI"/>
    <Setter Property="Foreground"
        Value="{DynamicResource TextPrimaryBrush}"/>
</Style>
```

Явный стиль (с x:Key)

Применяется только при явном указании:

```
<!-- Стиль окна из проекта -->
<Style x:Key="AppWindowStyle" TargetType="Window">
    <Setter Property="Background"
        Value="{DynamicResource WindowBackgroundBrush}"/>
    <Setter Property="Foreground"
        Value="{DynamicResource TextPrimaryBrush}"/>
    <Setter Property="WindowStyle" Value="None"/>
    <Setter Property="AllowsTransparency" Value="True"/>
    <Setter Property="ResizeMode" Value="CanResize"/>
</Style>

<!-- Применение -->
<Window Style="{DynamicResource AppWindowStyle}">
```

Setter — установщик свойств

Каждый Setter определяет пару Property/Value:

```
<Setter Property="Background" Value="#FFFFFF"/>
<Setter Property="Padding" Value="12,8"/>
<Setter Property="Cursor" Value="Hand"/>
<Setter Property="BorderThickness" Value="0,0,0,2"/>
```

BasedOn — наследование стилей

Стиль может наследовать свойства от другого стиля:

```
<Style x:Key="PrimaryButton" TargetType="Button"
    BasedOn="{StaticResource BaseButtonStyle}">
    <Setter Property="Background" Value="#1B3A5C"/>
    <Setter Property="Foreground" Value="White"/>
</Style>
```

ControlTemplate — полная замена визуального дерева

ControlTemplate позволяет полностью переопределить визуальную структуру элемента управления, сохраняя его поведение.

Пример: NavCategoryStyle — полная перешаблонизация RadioButton

В проекте категории навигации реализованы как RadioButton, полностью переопределённые через ControlTemplate:

```
<Style x:Key="NavCategoryStyle" TargetType="RadioButton">
    <Setter Property="Template">
        <Setter.Value>
            <ControlTemplate TargetType="RadioButton">
                <Border x:Name="border"
                    Background="Transparent"
                    BorderThickness="0,0,0,2"
                    BorderBrush="Transparent"
                    Padding="12,8"
                    Cursor="Hand">
                    <TextBlock x:Name="text"
                        Text="{TemplateBinding Content}"
                        FontSize="13"
                        Foreground=
                          "{DynamicResource TextSecondaryBrush}"
                        HorizontalAlignment="Center"/>
                </Border>
                <ControlTemplate.Triggers>
                    <Trigger Property="IsChecked" Value="True">
                        <Setter TargetName="border"
                            Property="BorderBrush"
                            Value="{DynamicResource AccentBrush}"/>
                        <Setter TargetName="text"
                            Property="Foreground"
                            Value="{DynamicResource AccentBrush}"/>
                        <Setter TargetName="text"
                            Property="FontWeight"
                            Value="SemiBold"/>
                    </Trigger>
                    <Trigger Property="IsMouseOver" Value="True">
                        <Setter TargetName="text"
                            Property="Foreground"
                            Value="{DynamicResource TextPrimaryBrush}"/>
                    </Trigger>
                </ControlTemplate.Triggers>
            </ControlTemplate>
        </Setter.Value>
    </Setter>
</Style>
```

Этот стиль превращает стандартный RadioButton (с кружком и текстом) в элемент навигации: прямоугольная область с текстом, подчёркивание акцентным цветом при выборе, смена цвета при наведении.

1.7 Триггеры (Trigger, MultiTrigger, DataTrigger, EventTrigger)

Триггеры позволяют динамически изменять свойства элементов при наступлении определённых условий — без написания кода в code-behind.

Property Trigger

Срабатывает при изменении значения свойства зависимости элемента:

```
<!-- Из AppWindowStyle: стиль при активации/деактивации окна -->
<Style x:Key="AppWindowStyle" TargetType="Window">
    <Style.Triggers>
        <Trigger Property="IsActive" Value="False">
            <Setter Property="Opacity" Value="0.95"/>
        </Trigger>
        <Trigger Property="IsActive" Value="True">
            <Setter Property="Opacity" Value="1.0"/>
        </Trigger>
    </Style.Triggers>
</Style>

<!-- Из NavCategoryStyle: стиль при выборе категории -->
<Trigger Property="IsChecked" Value="True">
    <Setter TargetName="border"
        Property="BorderBrush"
        Value="{DynamicResource AccentBrush}"/>
    <Setter TargetName="text"
        Property="FontWeight" Value="SemiBold"/>
</Trigger>
```

MultiTrigger

Срабатывает когда ВСЕ указанные условия истинны одновременно:

```
<!-- Из NavCategoryStyle: особый стиль при наведении
     НА ВЫБРАННУЮ категорию -->
<MultiTrigger>
    <MultiTrigger.Conditions>
        <Condition Property="IsMouseOver" Value="True"/>
        <Condition Property="IsChecked" Value="True"/>
    </MultiTrigger.Conditions>
    <MultiTrigger.Setters>
        <Setter TargetName="text" Property="Foreground"
            Value="{DynamicResource AccentHoverBrush}"/>
        <Setter TargetName="border" Property="Background"
            Value="{DynamicResource HoverBackgroundBrush}"/>
    </MultiTrigger.Setters>
</MultiTrigger>
```

DataTrigger

DataTrigger срабатывает на основе привязки данных, а не свойства элемента:

```
<!-- Показать/скрыть элемент на основе данных -->
<DataTrigger Binding="{Binding IsDownloaded}" Value="True">
    <Setter Property="Background" Value="#E8F5E9"/>
    <Setter Property="BorderBrush" Value="#4CAF50"/>
```

```
</DataTrigger>

<!-- Изменение текста кнопки по состоянию -->
<DataTrigger Binding="{Binding HasUpdates}" Value="True">
    <Setter TargetName="btnText"
        Property="Text" Value="Обновить"/>
</DataTrigger>
```

EventTrigger

EventTrigger запускает анимацию при наступлении события. В проекте используется для анимации баннера при загрузке:

```
<!-- Из FeaturedBannerView: анимация при загрузке -->
<UserControl.Triggers>
    <EventTrigger RoutedEvent="Loaded">
        <BeginStoryboard>
            <Storyboard>
                <ColorAnimation
                    Storyboard.TargetName="gradStop1"
                    Storyboard.TargetProperty="Color"
                    From="#FF6B35" To="#FF1744"
                    Duration="0:0:3"
                    AutoReverse="True"
                    RepeatBehavior="Forever"/>
                <ColorAnimation
                    Storyboard.TargetName="gradStop2"
                    Storyboard.TargetProperty="Color"
                    From="#1E88E5" To="#7C4DFF"
                    Duration="0:0:3"
                    AutoReverse="True"
                    RepeatBehavior="Forever"/>
            </Storyboard>
        </BeginStoryboard>
    </EventTrigger>
</UserControl.Triggers>
```

1.8 Анимации в WPF

WPF предоставляет встроенную систему анимаций, основанную на временных шкалах (Timeline). Анимации позволяют плавно изменять значения свойств элементов во времени.

Основные компоненты анимации

Компонент	Описание
Storyboard	Контейнер анимаций, управляющий временной шкалой
Timeline	Базовый класс всех анимаций
ColorAnimation	Анимация цвета (Color)
DoubleAnimation	Анимация чисел (Opacity, Width и др.)
ThicknessAnimation	Анимация отступов (Margin, Padding)
Duration	Длительность анимации (TimeSpan)
RepeatBehavior	Поведение повтора (Forever, 3x и др.)
AutoReverse	Автоматический возврат к исходному значению

ColorAnimation — анимация градиента FeaturedBannerView

В проекте AppStore баннер рекомендуемого приложения имеет анимированный градиентный фон:

```
<!-- FeaturedBannerView.xaml -->
<Border CornerRadius="16">
    <Border.Background>
        <LinearGradientBrush StartPoint="0,0" EndPoint="1,1">
            <GradientStop x:Name="gradStop1"
                Color="#FF6B35" Offset="0"/>
            <GradientStop x:Name="gradStop2"
                Color="#1E88E5" Offset="1"/>
        </LinearGradientBrush>
    </Border.Background>

    <!-- Содержимое баннера -->
</Border>

<!-- Анимация -->
<UserControl.Triggers>
    <EventTrigger RoutedEvent="Loaded">
        <BeginStoryboard>
            <Storyboard>
                <ColorAnimation
                    Storyboard.TargetName="gradStop1"
                    Storyboard.TargetProperty="Color"
                    From="#FF6B35" To="#FF1744"
                    Duration="0:0:3"
                    AutoReverse="True"
                    RepeatBehavior="Forever"/>
```

```
                <ColorAnimation
                    Storyboard.TargetName="gradStop2"
                    Storyboard.TargetProperty="Color"
                    From="#1E88E5" To="#7C4DFF"
                    Duration="0:0:3"
                    AutoReverse="True"
                    RepeatBehavior="Forever"/>
            </Storyboard>
        </BeginStoryboard>
    </EventTrigger>
</UserControl.Triggers>
```

Разбор анимации:

- **Storyboard.TargetName** — указывает на именованный элемент (GradientStop)
- **Storyboard.TargetProperty** — свойство, которое анимируется (Color)
- **From/To** — начальное и конечное значение цвета
- **Duration="0:0:3"** — длительность 3 секунды
- **AutoReverse="True"** — по достижении конечного значения анимация идёт обратно
- **RepeatBehavior="Forever"** — анимация повторяется бесконечно

В результате фон баннера плавно переливается между двумя наборами цветов, создавая визуально привлекательный эффект.

1.9 Ресурсы и словари ресурсов (ResourceDictionary)

Ресурсы в WPF — это объекты, определённые в словарях ресурсов и доступные для многократного использования. Система ресурсов обеспечивает темизацию, локализацию и переиспользование стилей.

Application.Resources и MergedDictionaries

Корневой словарь ресурсов приложения определяется в App.xaml:

```
<Application.Resources>
    <ResourceDictionary>
        <ResourceDictionary.MergedDictionaries>
            <!-- [0] Тема (светлая/тёмная) -->
            <ResourceDictionary
                Source="Resources/Styles/AppStyles.xaml"/>
            <!-- [1] Язык (ru/en) -->
            <ResourceDictionary
                Source="Resources/Localization/ru.xaml"/>
        </ResourceDictionary.MergedDictionaries>

        <!-- Глобальные ресурсы: конвертеры -->
        <converters:PriceConverter x:Key="PriceConv"/>
        <!-- ... -->
    </ResourceDictionary>
</Application.Resources>
```

StaticResource vs DynamicResource

Характеристика	StaticResource	DynamicResource
Когда разрешается	Однократно при загрузке XAML	При каждом обращении
Производительность	Быстрее	Немного медленнее
Изменения в runtime	Не отслеживает	Автоматически обновляет
Применение	Конвертеры, статические данные	Кисти тем, строки локализации

В проекте AppStore все кисти (Brush) и строки локализации используют DynamicResource, потому что они могут меняться при переключении темы или языка:

```
<!-- DynamicResource — обновится при смене темы -->
<TextBlock Foreground="{DynamicResource TextPrimaryBrush}"/>
<Border Background="{DynamicResource CardBackgroundBrush}"/>

<!-- StaticResource — не меняется -->
<TextBlock Text="{Binding Price,
    Converter={StaticResource PriceConv}}"/>
```

Переключение тем (ThemeService)

Переключение темы реализовано через замену первого словаря в MergedDictionaries:

```
public class ThemeService
{
    public void SetTheme(bool isDark)
    {
        var dict = new ResourceDictionary();
        dict.Source = new Uri(isDark
            ? "Resources/Styles/DarkTheme.xaml"
            : "Resources/Styles/AppStyles.xaml",
            UriKind.Relative);

        Application.Current.Resources
            .MergedDictionaries[0] = dict;
    }
}
```

Переключение языков (LocalizationService)

```
public class LocalizationService : ILocalizationService
{
    public void SetLanguage(string langCode)
    {
        var dict = new ResourceDictionary();
        dict.Source = new Uri(
            $"Resources/Localization/{langCode}.xaml",
            UriKind.Relative);

        Application.Current.Resources
            .MergedDictionaries[1] = dict;
    }
}
```

Кисти палитры (28+ DynamicResource)

Каждая тема определяет набор кистей SolidColorBrush. При переключении темы все элементы автоматически обновляются:

Ключ ресурса	Светлая тема	Тёмная тема	Назначение
WindowBackgroundBrush	#FFFFFF	#1E1E2E	Фон окна
CardBackgroundBrush	#F8F9FA	#2D2D3F	Фон карточек
TextPrimaryBrush	#1A1A2E	#E0E0E0	Основной текст
TextSecondaryBrush	#6C757D	#A0A0B0	Вторичный текст
AccentBrush	#1B3A5C	#5B9BD5	Акцентный цвет
AccentHoverBrush	#2A5580	#7BB3E0	Акцент при наведении
BorderBrush	#DEE2E6	#3D3D4F	Границы
SidebarBrush	#F0F2F5	#252535	Боковая панель
SearchBoxBrush	#FFFFFF	#2D2D3F	Поле поиска
HoverBackgroundBrush	#E9ECEF	#3D3D4F	Фон при наведении
SelectedItemBrush	#E3F2FD	#1E3A5F	Выбранный элемент
DividerBrush	#DEE2E6	#3D3D4F	Разделители

Ключ ресурса	Светлая тема	Тёмная тема	Назначение
SuccessBrush	#28A745	#4CAF50	Успех (скачано)
DangerBrush	#DC3545	#EF5350	Опасность (удаление)

1.10 Паттерн MVVM (Model-View-ViewModel)

MVVM (Model-View-ViewModel) — архитектурный паттерн, разработанный Microsoft для WPF-приложений. Он разделяет приложение на три слоя:

Слой	Ответственность	Примеры из проекта
Model	Данные и бизнес-логика	AppItem, User
View	Визуальное представление (XAML)	MainWindow, LoginWindow
ViewModel	Посредник между Model и View	MainViewModel, ProfileViewModel

Model — модели данных

AppItem — модель приложения

```
public class AppItem
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public string Name { get; set; } = "";
    public string Description { get; set; } = "";
    public string Developer { get; set; } = "";
    public string Category { get; set; } = "";
    public decimal Price { get; set; }
    public int Discount { get; set; }
    public double Rating { get; set; }
    public int DownloadCount { get; set; }
    public double SizeMb { get; set; }
    public string IconUrl { get; set; } = "";
    public string BannerUrl { get; set; } = "";
    public string AccentColor { get; set; } = "#1B3A5C";
    public List<string> Tags { get; set; } = new();
    public List<string> Screenshots { get; set; } = new();
    public bool IsDownloaded { get; set; }
    public bool IsFeatured { get; set; }
    public DateTime ReleaseDate { get; set; }
    public string Version { get; set; } = "1.0";
    public List<Guid> DownloadedByUsers { get; set; } = new();

    // Вычисляемое свойство
    public decimal FinalPrice =>
        Price * (1 - Discount / 100m);
}
```

User — модель пользователя

```
public enum UserRole { User, Admin }

public class User
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public string Login { get; set; } = "";
    public string PasswordHash { get; set; } = "";
    public string FirstName { get; set; } = "";
    public string LastName { get; set; } = "";
    public string Email { get; set; } = "";
    public UserRole Role { get; set; } = UserRole.User;

    // Вычисляемое свойство — первая буква имени
    public string AvatarLetter =>
        string.IsNullOrEmpty(FirstName)
            ? "?"
            : FirstName[0].ToString().ToUpper();
}
```

View — представления

Представления — это XAML-файлы, описывающие визуальную структуру. В проекте:

- **MainWindow.xaml** — основное окно с навигацией, списком и деталями приложения
- **LoginWindow.xaml** — окно авторизации
- **ProfilePopup.xaml** — всплывающее окно профиля с настройками темы и языка
- **FeaturedBannerView.xaml** — баннер рекомендуемого приложения с анимацией
- **AppCardView.xaml** — карточка приложения (плиточный вид)
- **AppRowView.xaml** — строка приложения (списочный вид)
- **AppDetailView.xaml** — подробная информация о приложении
- **AddEditWindow.xaml** — диалог добавления/редактирования приложения

ViewModel — модели представления

ViewModelBase — базовый класс

Все ViewModel наследуют от ViewModelBase, который реализует INotifyPropertyChanged (см. раздел 1.3).

Связь View и ViewModel

```
// В code-behind MainWindow.xaml.cs
public MainWindow(MainViewModel viewModel)
{
    InitializeComponent();
    DataContext = viewModel;  // Устанавливаем ViewModel
                              // как контекст данных
}

// В App.xaml.cs при запуске
var appRepo = new JsonAppRepository();
var appService = new AppService(appRepo);
var vm = new MainViewModel(appService, authService,
    themeService, locService);
var mainWindow = new MainWindow(vm);
mainWindow.Show();
```

Преимущества MVVM

- **Тестируемость** — ViewModel тестируется без UI (unit tests)
- **Разделение ответственности** — дизайнер работает с XAML, разработчик с C#
- **Переиспользование** — ViewModel может использоваться разными View
- **Поддерживаемость** — изменения UI не затрагивают логику и наоборот

1.11 Паттерн Repository

Repository (Репозиторий) — паттерн проектирования, который абстрагирует слой доступа к данным. Он предоставляет единый интерфейс для работы с данными, скрывая детали хранения (файл, база данных, API).

Интерфейс IAppRepository

```
public interface IAppRepository
{
    List<AppItem> GetAll();
    AppItem? GetById(Guid id);
    void Add(AppItem app);
    void Update(AppItem app);
    void Delete(Guid id);
    void Download(Guid id);
    void Uninstall(Guid id);
    void RestoreDefaults();
}
```

Интерфейс IUserRepository

```
public interface IUserRepository
{
    List<User> GetAll();
    User? GetByLogin(string login);
    void Add(User user);
    void Update(User user);
}
```

JsonAppRepository — хранение в JSON-файле

Реализация репозитория с кэшированием в памяти и сохранением в JSON-файл:

```
public class JsonAppRepository : IAppRepository
{
    private readonly string _filePath;
    private List<AppItem> _cache;

    public JsonAppRepository()
    {
        var appData = Environment.GetFolderPath(
            Environment.SpecialFolder.ApplicationData);
        var dir = Path.Combine(appData, "AppStore");
        Directory.CreateDirectory(dir);
        _filePath = Path.Combine(dir, "apps.json");
        _cache = Load();
    }

    private List<AppItem> Load()
    {
        if (!File.Exists(_filePath))
        {
            var seed = SeedData();
            Save(seed);
            return seed;
        }
        var json = File.ReadAllText(_filePath);
        return JsonConvert.DeserializeObject<List<AppItem>>(json)
```

```
            ?? SeedData();
    }

    private void Save(List<AppItem> apps)
    {
        var json = JsonConvert.SerializeObject(apps,
            Formatting.Indented);
        File.WriteAllText(_filePath, json);
    }

    public List<AppItem> GetAll() => new(_cache);

    public AppItem? GetById(Guid id)
        => _cache.FirstOrDefault(a => a.Id == id);

    public void Add(AppItem app)
    { _cache.Add(app); Save(_cache); }

    public void Update(AppItem app)
    {
        var idx = _cache.FindIndex(a => a.Id == app.Id);
        if (idx >= 0) { _cache[idx] = app; Save(_cache); }
    }

    public void Delete(Guid id)
    { _cache.RemoveAll(a => a.Id == id); Save(_cache); }

    public void Download(Guid id) { /* ... */ }
    public void Uninstall(Guid id) { /* ... */ }

    public void RestoreDefaults()
    { _cache = SeedData(); Save(_cache); }
}
```

EfAppRepository — Entity Framework Core

Альтернативная реализация с использованием EF Core и PostgreSQL:

```
public class EfAppRepository : IAppRepository
{
    private readonly AppStoreDbContext _context;

    public EfAppRepository(AppStoreDbContext context)
    {
        _context = context;
        _context.Database.EnsureCreated();
    }

    public List<AppItem> GetAll()
        => _context.Apps.ToList();

    public AppItem? GetById(Guid id)
        => _context.Apps.Find(id);

    public void Add(AppItem app)
    {
        _context.Apps.Add(app);
        _context.SaveChanges();
    }

    public void Update(AppItem app)
    {
        _context.Apps.Update(app);
        _context.SaveChanges();
```

```
    }

    public void Delete(Guid id)
    {
        var app = _context.Apps.Find(id);
        if (app != null)
        {
            _context.Apps.Remove(app);
            _context.SaveChanges();
        }
    }
    // ...
}
```

Переключение реализаций (инверсия зависимости)

Благодаря интерфейсам, переключение между JSON-хранилищем и базой данных — это замена одной строки:

```
// Вариант 1: JSON-файл (по умолчанию)
IAppRepository repo = new JsonAppRepository();

// Вариант 2: PostgreSQL через EF Core
var context = new AppStoreDbContext();
IAppRepository repo = new EfAppRepository(context);

// Далее код работает одинаково
var service = new AppService(repo);
```

1.12 Сервисный слой (Service Layer)

Сервисный слой инкапсулирует бизнес-логику приложения, являясь посредником между ViewModel и слоем данных (Repository).

IAppService / AppService

```
public interface IAppService
{
    List<AppItem> GetAll();
    List<AppItem> Search(string query);
    List<AppItem> Filter(string? category, string? priceRange,
        double? minRating, bool? downloaded, string? sortBy);
    List<AppItem> GetFeatured();
    AppItem? GetById(Guid id);
    void Add(AppItem app);
    void Update(AppItem app);
    void Delete(Guid id);
    void Download(Guid id);
    void Uninstall(Guid id);
    void RestoreDefaults();
}

public class AppService : IAppService
{
    private readonly IAppRepository _repo;

    public AppService(IAppRepository repo) => _repo = repo;

    public List<AppItem> Search(string query)
    {
        if (string.IsNullOrWhiteSpace(query))
            return _repo.GetAll();

        var q = query.ToLower();
        return _repo.GetAll().Where(a =>
            a.Name.ToLower().Contains(q) ||
            a.Description.ToLower().Contains(q) ||
            a.Developer.ToLower().Contains(q) ||
            a.Tags.Any(t => t.ToLower().Contains(q))
        ).ToList();
    }

    public List<AppItem> Filter(string? category,
        string? priceRange, double? minRating,
        bool? downloaded, string? sortBy)
    {
        var apps = _repo.GetAll().AsEnumerable();

        if (!string.IsNullOrEmpty(category)
            && category != "Все")
            apps = apps.Where(a => a.Category == category);

        if (priceRange == "free")
            apps = apps.Where(a => a.Price == 0);
        else if (priceRange == "paid")
            apps = apps.Where(a => a.Price > 0);

        if (minRating.HasValue)
            apps = apps.Where(a => a.Rating >= minRating);

        if (downloaded.HasValue)
            apps = apps.Where(a =>
                a.IsDownloaded == downloaded);

        apps = sortBy switch
        {
            "name" => apps.OrderBy(a => a.Name),
            "price" => apps.OrderBy(a => a.FinalPrice),
            "rating" => apps.OrderByDescending(a => a.Rating),
```

```
            "downloads" => apps.OrderByDescending(
                a => a.DownloadCount),
            "date" => apps.OrderByDescending(
                a => a.ReleaseDate),
            _ => apps
        };

        return apps.ToList();
    }

    public List<AppItem> GetFeatured()
        => _repo.GetAll().Where(a => a.IsFeatured).ToList();
}
```

IAuthService / AuthService

```
public interface IAuthService
{
    User? CurrentUser { get; }
    bool Login(string login, string password);
    void Logout();
    void UpdateProfile(string firstName, string lastName,
        string email);
    bool ChangePassword(string oldPassword, string newPassword);
}

public class AuthService : IAuthService
{
    private readonly IUserRepository _userRepo;
    public User? CurrentUser { get; private set; }

    public AuthService(IUserRepository userRepo)
        => _userRepo = userRepo;

    public bool Login(string login, string password)
    {
        var hash = HashPassword(password);
        var user = _userRepo.GetByLogin(login);
        if (user != null && user.PasswordHash == hash)
        {
            CurrentUser = user;
            return true;
        }
        return false;
    }

    public void Logout() => CurrentUser = null;

    public bool ChangePassword(string oldPwd, string newPwd)
    {
        if (CurrentUser == null) return false;
        if (CurrentUser.PasswordHash != HashPassword(oldPwd))
            return false;
        CurrentUser.PasswordHash = HashPassword(newPwd);
        _userRepo.Update(CurrentUser);
        return true;
    }

    private static string HashPassword(string password)
    {
        var bytes = System.Text.Encoding.UTF8
            .GetBytes(password);
        var hash = System.Security.Cryptography.SHA256
            .HashData(bytes);
        return Convert.ToHexString(hash).ToLower();
    }
}
```

ThemeService

```
public class ThemeService
{
    private bool _isDark;

    public bool IsDark => _isDark;

    public void SetTheme(bool isDark)
    {
        _isDark = isDark;
        var dict = new ResourceDictionary
        {
            Source = new Uri(isDark
                ? "Resources/Styles/DarkTheme.xaml"
                : "Resources/Styles/AppStyles.xaml",
                UriKind.Relative)
        };
        Application.Current.Resources
            .MergedDictionaries[0] = dict;
    }
}
```

LocalizationService

```
public interface ILocalizationService
{
    void SetLanguage(string langCode);
    string CurrentLanguage { get; }
}

public class LocalizationService : ILocalizationService
{
    public string CurrentLanguage { get; private set; } = "ru";

    public void SetLanguage(string langCode)
    {
        CurrentLanguage = langCode;
        var dict = new ResourceDictionary
        {
            Source = new Uri(
                $"Resources/Localization/{langCode}.xaml",
                UriKind.Relative)
        };
        Application.Current.Resources
            .MergedDictionaries[1] = dict;
    }
}
```

1.13 Entity Framework Core + PostgreSQL

Entity Framework Core (EF Core) — ORM-фреймворк (Object-Relational Mapping), который позволяет работать с базой данных через объекты C#, не написав ни одного SQL-запроса вручную.

Что такое ORM

ORM автоматически преобразует:

- Классы C# → таблицы базы данных
- Свойства → столбцы
- Объекты → строки
- LINQ-запросы → SQL-запросы

AppStoreDbContext

```
public class AppStoreDbContext : DbContext
{
    public DbSet<AppItem> Apps { get; set; }
    public DbSet<User> Users { get; set; }

    protected override void OnConfiguring(
        DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseNpgsql(
            "Host=localhost;Port=5432;" +
            "Database=appstore;" +
            "Username=postgres;Password=postgres");
    }

    protected override void OnModelCreating(
        ModelBuilder modelBuilder)
    {
        // AppItem configuration
        modelBuilder.Entity<AppItem>(entity =>
        {
            entity.HasKey(a => a.Id);

            entity.Property(a => a.Name)
                .IsRequired()
                .HasMaxLength(200);

            entity.Property(a => a.Price)
                .HasColumnType("decimal(10,2)");

            // Конвертация List<string> → строка с разделителем
            entity.Property(a => a.Tags)
                .HasConversion(
                    v => string.Join(",", v),
                    v => v.Split(",",
                        StringSplitOptions.RemoveEmptyEntries)
                        .ToList()
                );

            entity.Property(a => a.Screenshots)
                .HasConversion(
                    v => string.Join("|", v),
                    v => v.Split("|",
                        StringSplitOptions.RemoveEmptyEntries)
                        .ToList()
                );

            entity.Property(a => a.DownloadedByUsers)
                .HasConversion(
                    v => string.Join(",", v),
                    v => v.Split(",",
                        StringSplitOptions.RemoveEmptyEntries)
                        .Select(Guid.Parse).ToList()
                );

            // Игнорировать вычисляемые свойства
            entity.Ignore(a => a.FinalPrice);
        });

        // User configuration
        modelBuilder.Entity<User>(entity =>
        {
            entity.HasKey(u => u.Id);
            entity.Property(u => u.Login).IsRequired();
```

```
            entity.Property(u => u.PasswordHash).IsRequired();
            entity.Ignore(u => u.AvatarLetter);
        });
    }
}
```

Fluent API — конфигурация модели

В OnModelCreating используется Fluent API для настройки маппинга:

Метод	Описание	Пример
HasKey	Определяет первичный ключ	entity.HasKey(a => a.Id)
Property	Настройка столбца	entity.Property(a => a.Name)
IsRequired	NOT NULL	.IsRequired()
HasMaxLength	Макс. длина VARCHAR	.HasMaxLength(200)
HasColumnType	Тип столбца SQL	.HasColumnType("decimal(10,2)")
HasConversion	Конвертация при чтении/записи	List → comma-separated string
Ignore	Не создавать столбец	entity.Ignore(a => a.FinalPrice)

NuGet-пакеты для EF Core

```
<!-- AppStore.csproj -->
<ItemGroup>
    <PackageReference
        Include="Microsoft.EntityFrameworkCore"
        Version="8.0.0" />
    <PackageReference
        Include="Microsoft.EntityFrameworkCore.Design"
        Version="8.0.0" />
    <PackageReference
        Include="Npgsql.EntityFrameworkCore.PostgreSQL"
        Version="8.0.0" />
</ItemGroup>
```

EnsureCreated vs Migrations

В проекте используется **EnsureCreated()**, который создаёт базу данных и таблицы при первом запуске, если они не существуют. Это удобно для прототипирования, но в продакшене рекомендуется использовать миграции (Migrations), которые позволяют эволюционировать схему БД.

1.14 Хеширование паролей (SHA256)

Хеширование — это однонаправленное преобразование данных фиксированной длины. В проекте AppStore используется алгоритм SHA-256 для безопасного хранения паролей.

Зачем хешировать пароли

- Пароль не хранится в открытом виде — даже при утечке данных злоумышленник не получит пароль
- SHA-256 — однонаправленная функция: нельзя получить пароль из хеша
- Одинаковый вход всегда даёт одинаковый хеш — позволяет сравнивать пароли

Реализация в проекте

```
using System.Security.Cryptography;
using System.Text;

private static string HashPassword(string password)
{
    // 1. Преобразуем строку в байты UTF-8
    var bytes = Encoding.UTF8.GetBytes(password);

    // 2. Вычисляем SHA-256 хеш
    var hash = SHA256.HashData(bytes);

    // 3. Преобразуем байты хеша в hex-строку
    return Convert.ToHexString(hash).ToLower();
}

// Примеры:
// "admin"  → "8c6976e5b5410415bde908bd4dee15dfb167a9c873fc4bb8a81f6f2ab448a918"
// "1234"   → "03ac674216f3e15c761ee1a5e255f067953623c8b388b4459e13f978d7c846f4"
```

Процесс аутентификации

1. При регистрации пароль хешируется и хеш сохраняется в базу
2. При входе введённый пароль хешируется тем же алгоритмом
3. Хеш введённого пароля сравнивается с хешем в базе
4. Если хеши совпадают — пароль верный

```
// Seed-данные: предустановленные пользователи
new User
{
    Login = "admin",
    PasswordHash = HashPassword("admin"),  // SHA256("admin")
    Role = UserRole.Admin
},
new User
{
    Login = "user",
    PasswordHash = HashPassword("1234"),   // SHA256("1234")
    Role = UserRole.User
}
```

1.15 Сериализация JSON (Newtonsoft.Json)

Сериализация — процесс преобразования объекта C# в текстовый формат (JSON) для сохранения или передачи. Десериализация — обратный процесс.

Основные методы

```
using Newtonsoft.Json;

// Сериализация: объект → JSON строка
string json = JsonConvert.SerializeObject(appsList,
    Formatting.Indented);

// Десериализация: JSON строка → объект
List<AppItem> apps = JsonConvert
    .DeserializeObject<List<AppItem>>(json);
```

Хранение данных в %AppData%

Данные приложения хранятся в директории %AppData%/AppStore/:

```
var appData = Environment.GetFolderPath(
    Environment.SpecialFolder.ApplicationData);
var dir = Path.Combine(appData, "AppStore");
Directory.CreateDirectory(dir);

// Файлы:
// %AppData%/AppStore/apps.json  — приложения
// %AppData%/AppStore/users.json — пользователи
```

Formatting.Indented

Параметр Formatting.Indented форматирует JSON с отступами, что удобно для отладки и чтения:

```
// Без форматирования:
{"Name":"VS Code","Price":0,"Rating":4.8}

// С Formatting.Indented:
{
  "Name": "VS Code",
  "Price": 0,
  "Rating": 4.8
}
```

1.16 Кастомный курсор (.cur файл)

WPF позволяет использовать кастомные курсоры мыши через файлы формата .cur (статический курсор) или .ani (анимированный).

Формат .cur файла

Файл .cur содержит растровое изображение курсора с прозрачностью и информацию о "горячей точке" (hot spot) — координату, считающуюся позицией клика. В проекте используется файл Resources/Cursors/arrow.cur.

Использование в XAML

```
<!-- В стилях элементов -->
<Setter Property="Cursor" Value="Hand"/>

<!-- Встроенные курсоры WPF: -->
<!-- Arrow, Hand, Wait, IBeam, Cross, SizeNS и др. -->

<!-- Кастомный курсор из ресурсов: -->
<Setter Property="Cursor">
    <Setter.Value>
        <Cursor>Resources/Cursors/arrow.cur</Cursor>
    </Setter.Value>
</Setter>
```

Cursor="Hand" широко используется в проекте для интерактивных элементов — кнопок навигации, карточек приложений, ссылок.

ЧАСТЬ 2

Архитектура проекта

2.1 Структура файлов проекта

Проект AppStore организован по принципу разделения ответственности. Каждая папка содержит файлы определённого назначения:

```
AppStore/
├── Models/
│   ├── AppItem.cs              — Модель приложения (20+ свойств)
│   └── User.cs                 — Модель пользователя + UserRole
├── Data/
│   ├── IAppRepository.cs       — Интерфейс репозитория приложений
│   ├── IUserRepository.cs      — Интерфейс репозитория пользователей
│   ├── JsonAppRepository.cs    — JSON-реализация (файл + кэш)
│   ├── JsonUserRepository.cs   — JSON-реализация для Users
│   ├── AppStoreDbContext.cs     — EF Core DbContext (PostgreSQL)
│   ├── EfAppRepository.cs      — EF Core реализация для Apps
│   └── EfUserRepository.cs     — EF Core реализация для Users
├── Services/
│   ├── IAppService.cs          — Интерфейс сервиса приложений
│   ├── AppService.cs           — Поиск, фильтрация, сортировка
│   ├── IAuthService.cs         — Интерфейс аутентификации
│   ├── AuthService.cs          — Логин, SHA256, смена пароля
│   ├── ILocalizationService.cs — Интерфейс локализации
│   ├── LocalizationService.cs  — Переключение языков
│   └── ThemeService.cs         — Переключение тем
├── Commands/
│   ├── RelayCommand.cs         — RelayCommand + RelayCommand<T>
│   └── AppCommands.cs          — DownloadAppCommand, DeleteAppCommand
├── Converters/
│   └── Converters.cs           — Все 17 конвертеров значений
├── ViewModels/
│   ├── ViewModelBase.cs        — INotifyPropertyChanged + SetField
│   ├── MainViewModel.cs        — Главная ViewModel (11+ команд)
│   ├── ProfileViewModel.cs     — ViewModel профиля пользователя
│   └── AddEditAppViewModel.cs  — ViewModel диалога добавления
├── Views/
│   ├── MainWindow.xaml/.cs     — Главное окно приложения
│   ├── LoginWindow.xaml/.cs    — Окно авторизации
│   ├── ProfilePopup.xaml/.cs   — Всплывающее окно профиля
│   ├── FeaturedBannerView.xaml/.cs — Анимированный баннер
│   ├── AppDetailView.xaml/.cs  — Детали приложения
│   ├── AppCardView.xaml/.cs    — Карточка (плиточный вид)
│   ├── AppRowView.xaml/.cs     — Строка (списочный вид)
│   ├── AddEditWindow.xaml/.cs  — Диалог добавления/редактирования
│   ├── EditDeleteDialog.xaml/.cs — Диалог подтверждения
│   └── ConfirmDialog.xaml/.cs  — Диалог подтверждения действий
├── Resources/
│   ├── Styles/
│   │   ├── AppStyles.xaml      — Светлая тема (28+ кистей)
│   │   └── DarkTheme.xaml      — Тёмная тема
│   ├── Localization/
│   │   ├── en.xaml             — Английский язык
```

```
│   │   └── ru.xaml             — Русский язык
│   ├── Cursors/
│   │   └── arrow.cur           — Кастомный курсор
│   └── Buffer/                 — JSON буфер данных
├── App.xaml                    — Ресурсы приложения + конвертеры
├── App.xaml.cs                 — Точка входа + DI
├── AppStore.csproj             — Файл проекта + NuGet
├── AssemblyInfo.cs             — Метаданные сборки
└── AppStore.slnx               — Файл решения
```

2.2 Модели данных

AppItem — модель приложения

AppItem представляет приложение в магазине. Содержит 20+ свойств:

Свойство	Тип	Описание
Id	Guid	Уникальный идентификатор
Name	string	Название приложения
Description	string	Описание
Developer	string	Разработчик
Category	string	Категория (Игры, Утилиты и др.)
Price	decimal	Цена (0 = бесплатно)
Discount	int	Скидка в процентах (0-100)
Rating	double	Рейтинг (0.0-5.0)
DownloadCount	int	Количество скачиваний
SizeMb	double	Размер в мегабайтах
IconUrl	string	URL иконки
BannerUrl	string	URL баннера
AccentColor	string	Акцентный цвет (#hex)
Tags	List<string>	Теги / ключевые слова
Screenshots	List<string>	URL скриншотов
IsDownloaded	bool	Скачано ли приложение
IsFeatured	bool	Рекомендуемое приложение
ReleaseDate	DateTime	Дата выхода
Version	string	Версия
DownloadedByUsers	List<Guid>	ID пользователей, скачавших
FinalPrice	decimal	Вычисляемое: Price * (1 - Discount/100)

User — модель пользователя

Свойство	Тип	Описание
Id	Guid	Уникальный идентификатор
Login	string	Логин для входа
PasswordHash	string	SHA-256 хеш пароля
FirstName	string	Имя
LastName	string	Фамилия
Email	string	Электронная почта
Role	UserRole	Роль: User или Admin
AvatarLetter	string	Вычисляемое: первая буква имени

Перечисление UserRole

```
public enum UserRole
{
    User,   // Обычный пользователь
    Admin   // Администратор (может добавлять/удалять приложения)
}
```

2.3 Слой данных (Data Layer)

Интерфейсы репозиториев

Два интерфейса определяют контракт доступа к данным:

Метод IAppRepository	Описание
GetAll()	Получить все приложения
GetById(Guid id)	Найти приложение по ID
Add(AppItem app)	Добавить новое приложение
Update(AppItem app)	Обновить существующее
Delete(Guid id)	Удалить по ID
Download(Guid id)	Отметить как скачанное
Uninstall(Guid id)	Отметить как удалённое
RestoreDefaults()	Восстановить начальные данные

Метод IUserRepository	Описание
GetAll()	Получить всех пользователей
GetByLogin(string login)	Найти по логину
Add(User user)	Добавить пользователя
Update(User user)	Обновить данные пользователя

JsonAppRepository — детали реализации

- **Хранение:** JSON-файл в %AppData%/AppStore/apps.json
- **Кэш:** List<AppItem> в памяти, синхронизируется при каждой записи
- **Load/Save:** чтение при создании, запись после каждой мутации
- **SeedData:** 12 предустановленных приложений (VS Code, Photoshop, Spotify и др.)

JsonUserRepository — предустановленные пользователи

```
private List<User> SeedData() => new()
{
    new User
    {
        Login = "admin",
        PasswordHash = HashPassword("admin"),
        FirstName = "Администратор",
        LastName = "",
        Email = "admin@appstore.local",
        Role = UserRole.Admin
    },
    new User
    {
        Login = "user",
        PasswordHash = HashPassword("1234"),
        FirstName = "Пользователь",
        LastName = "Тестовый",
        Email = "user@appstore.local",
        Role = UserRole.User
    }
};
```

EF Core — конвертация сложных типов

EF Core не поддерживает List<string> и List<Guid> напрямую. Используется HasConversion для преобразования:

Свойство	C# тип	SQL столбец	Конвертация
Tags	List<string>	TEXT	Join/Split по запятой
Screenshots	List<string>	TEXT	Join/Split по |
DownloadedByUsers	List<Guid>	TEXT	Join/Split + Guid.Parse

2.4 Сервисный слой

AppService — бизнес-логика приложений

Операция	Логика
Search(query)	Поиск по Name, Description, Developer, Tags (регистронезависимый)
Filter(category, price, rating, downloaded, sort)	Цепочка фильтров + сортировка
GetFeatured()	Возвращает приложения с IsFeatured=true
GetAll(), GetById(), Add(), Update(), Delete()	Делегирование в Repository
Download(id), Uninstall(id)	Делегирование в Repository
RestoreDefaults()	Сброс к начальным данным (12 приложений)

AuthService — аутентификация

Операция	Логика
Login(login, pwd)	SHA256(pwd) → сравнение с PasswordHash в БД
Logout()	CurrentUser = null
UpdateProfile(first, last, email)	Обновление данных + сохранение через Repository
ChangePassword(old, new)	Проверка старого + хеширование нового

ThemeService — управление темой

Метод `SetTheme(bool isDark)` заменяет `MergedDictionaries[0]` на соответствующий ResourceDictionary. Все элементы, использующие DynamicResource, обновляются автоматически.

LocalizationService — управление языком

Метод `SetLanguage(string langCode)` заменяет `MergedDictionaries[1]`. Поддерживаемые языки: ru (русский), en (английский). PriceConverter адаптирует символ валюты (₽/\$).

2.5 ViewModels

ViewModelBase

Базовый класс с реализацией INotifyPropertyChanged и вспомогательным методом SetField<T> (см. раздел 1.3).

MainViewModel — главная ViewModel

Самый крупный ViewModel проекта, управляющий всем основным окном:

Коллекции данных

Свойство	Тип	Описание
Apps	ObservableCollection<AppItem>	Отображаемый список приложений
FeaturedApps	ObservableCollection<AppItem>	Рекомендуемые приложения
AllTags	ObservableCollection<string>	Все теги для фильтрации

Состояние UI

Свойство	Тип	Описание
SelectedApp	AppItem?	Выбранное приложение
SearchQuery	string	Текст поиска (авто-обновление)
SelectedCategory	string	Выбранная категория
SelectedSort	string	Критерий сортировки
IsDetailView	bool	Показана ли детальная страница
IsGridView	bool	Плиточный (true) или списочный вид

Команды (11+)

Команда	Действие
SearchCommand	Поиск по всем полям
SelectAppCommand	Открыть детали приложения
BackCommand	Вернуться к списку
DownloadCommand	Скачать/удалить приложение
DeleteCommand	Удалить из каталога (Admin)
AddAppCommand	Открыть диалог добавления
EditAppCommand	Открыть диалог редактирования
CategoryCommand	Фильтрация по категории
SortCommand	Изменить сортировку
ToggleViewCommand	Переключить плитки/список
OpenProfileCommand	Открыть окно профиля
RestoreDefaultsCommand	Сбросить к начальным данным

Событие LogoutRequested

MainViewModel определяет событие LogoutRequested, которое используется в App.xaml.cs для перезапуска цикла авторизации:

```
public event EventHandler? LogoutRequested;

// Вызов при нажатии кнопки "Выйти"
private void OnLogout(object? param)
{
    _authService.Logout();
    LogoutRequested?.Invoke(this, EventArgs.Empty);
}
```

ProfileViewModel

Управляет окном профиля пользователя:

Свойство/Команда	Описание
FirstName / LastName / Email	Редактируемые поля профиля (TwoWay binding)
IsDarkMode	Переключатель темы
CurrentLanguage	Выбранный язык
SaveProfileCommand	Сохранить изменения профиля
ChangePasswordCommand	Сменить пароль
ToggleThemeCommand	Переключить тему
SetLanguageCommand	Установить язык
Func<string> PasswordProvider	Делегат для получения пароля из PasswordBox

Примечание: PasswordBox не поддерживает привязку данных из соображений безопасности. Поэтому используется делегат Func<string> для получения пароля из code-behind.

AddEditAppViewModel

ViewModel для диалога добавления и редактирования приложений. Содержит все свойства AppItem как редактируемые поля с двусторонней привязкой и команду сохранения.

2.6 Команды

RelayCommand / RelayCommand<T>

Универсальные реализации ICommand на основе делегатов (см. раздел 1.5). Используются для создания команд прямо в ViewModel без отдельных классов:

```
// В MainViewModel
SearchCommand = new RelayCommand(_ =>
    RefreshApps());

SelectAppCommand = new RelayCommand(param =>
{
    if (param is AppItem app)
    {
        SelectedApp = app;
        IsDetailView = true;
    }
});

BackCommand = new RelayCommand(_ =>
{
    SelectedApp = null;
    IsDetailView = false;
```

```
});
```

DownloadAppCommand — специализированная команда

Инкапсулирует логику скачивания/удаления приложения. Принимает IAppRepository для прямого доступа к данным и Action callback для обновления UI после операции.

DeleteAppCommand — специализированная команда

Инкапсулирует удаление приложения из каталога. Принимает Guid как CommandParameter и вызывает callback с ID удалённого приложения для очистки коллекций в ViewModel.

2.7 Конвертеры (все 17)

Все конвертеры определены в файле Converters/Converters.cs и зарегистрированы в App.xaml:

#	Конвертер	Вход	Выход	Логика
1	BoolToVisibilityConverter	bool	Visibility	true→Visible, false→Collapsed
2	InverseBoolToVisibilityConverter	bool	Visibility	true→Collapsed, false→Visible
3	NullToVisibilityConverter	object	Visibility	null→Collapsed, иначе→Visible
4	StringNotEmptyToVisibilityConverter	string	Visibility	empty→Collapsed, иначе→Visible
5	PriceConverter	decimal	string	0→Бесплатно/Free + валюта (₽/$)
6	AppGetLabelConverter	bool	string	IsDownloaded: Открыть/Установить
7	DownloadButtonTextConverter	bool	string	Текст кнопки загрузки
8	DownloadCountConverter	int	string	≥1M→XM, ≥1K→XK, иначе число
9	RatingToStarsConverter	double	string	4.5→★★★★½☆
10	SizeConverter	double	string	150.5→150.5 MB
11	SizeNumberConverter	double	string	150.5→150.5
12	SizeUnitConverter	double	string	≥1024→GB, иначе→MB
13	StringToColorBrushConverter	string	Brush	#hex→SolidColorBrush
14	DiscountVisibilityConverter	int	Visibility	>0→Visible, 0→Collapsed
15	DiscountWithPriceVisibilityConverter	object[]	Visibility	IMultiValueConverter: discount>0 AND price>0
16	StockToColorConverter	int	Brush	Цвет по количеству на складе
17	RoleNameConverter	UserRole	string	Admin→Администратор, User→Пользователь

2.8 Запуск приложения (App.xaml.cs)

App.xaml.cs — точка входа приложения, где происходит инициализация всех зависимостей и управление жизненным циклом:

```
public partial class App : Application
{
    private static ThemeService _themeService = new();

    protected override void OnStartup(StartupEventArgs e)
    {
        base.OnStartup(e);
        ShowLogin();
    }

    private void ShowLogin()
    {
        // 1. Создаём репозиторий пользователей
        var userRepo = new JsonUserRepository();

        // 2. Создаём сервис аутентификации
        var authService = new AuthService(userRepo);

        // 3. Показываем окно логина
        var loginWindow = new LoginWindow(authService);
        var result = loginWindow.ShowDialog();

        if (result == true && authService.CurrentUser != null)
        {
            // 4. После успешного логина — создаём всё остальное
            var appRepo = new JsonAppRepository();
            var appService = new AppService(appRepo);
            var locService = new LocalizationService();

            // 5. Создаём главную ViewModel
            var vm = new MainViewModel(
                appService, authService,
                _themeService, locService);

            // 6. Подписываемся на выход
            vm.LogoutRequested += (s, e) =>
            {
                Application.Current.Windows
                    .OfType<MainWindow>()
                    .FirstOrDefault()?.Close();
                ShowLogin();  // Перезапуск цикла
            };

            // 7. Показываем главное окно
            var mainWindow = new MainWindow(vm);
            mainWindow.Show();
        }
        else
        {
            Shutdown();
        }
    }
}
```

Ключевые моменты:

- **ThemeService — static**: сохраняет выбранную тему между логинами/логаутами
- **ShowLogin()**: рекурсивно вызывается при LogoutRequested

- **ShowDialog()**: LoginWindow блокирует выполнение до закрытия
- **Каскадное создание**: UserRepo → AuthService → Login → AppRepo → AppService → VM → Window

2.9 Тёмная/Светлая тема

Приложение поддерживает две темы: светлую (по умолчанию) и тёмную. Переключение происходит мгновенно без перезагрузки.

Механизм работы

1. Каждая тема — отдельный ResourceDictionary с набором кистей (SolidColorBrush)
2. Все элементы UI ссылаются на кисти через DynamicResource
3. ThemeService.SetTheme() заменяет MergedDictionaries[0]
4. DynamicResource автоматически подхватывает новые значения

Файлы тем

AppStyles.xaml (светлая тема) — фрагмент

```
<ResourceDictionary>
    <!-- Основные цвета -->
    <SolidColorBrush x:Key="WindowBackgroundBrush"
        Color="#FFFFFF"/>
    <SolidColorBrush x:Key="CardBackgroundBrush"
        Color="#F8F9FA"/>
    <SolidColorBrush x:Key="TextPrimaryBrush"
        Color="#1A1A2E"/>
    <SolidColorBrush x:Key="TextSecondaryBrush"
        Color="#6C757D"/>
    <SolidColorBrush x:Key="AccentBrush"
        Color="#1B3A5C"/>
    <SolidColorBrush x:Key="BorderBrush"
        Color="#DEE2E6"/>
    <SolidColorBrush x:Key="SidebarBrush"
        Color="#F0F2F5"/>
    <SolidColorBrush x:Key="HoverBackgroundBrush"
        Color="#E9ECEF"/>
    <SolidColorBrush x:Key="SelectedItemBrush"
        Color="#E3F2FD"/>
    <!-- + стили элементов -->
</ResourceDictionary>
```

DarkTheme.xaml (тёмная тема) — фрагмент

```
<ResourceDictionary>
    <SolidColorBrush x:Key="WindowBackgroundBrush"
        Color="#1E1E2E"/>
    <SolidColorBrush x:Key="CardBackgroundBrush"
        Color="#2D2D3F"/>
    <SolidColorBrush x:Key="TextPrimaryBrush"
        Color="#E0E0E0"/>
    <SolidColorBrush x:Key="TextSecondaryBrush"
        Color="#A0A0B0"/>
    <SolidColorBrush x:Key="AccentBrush"
        Color="#5B9BD5"/>
    <SolidColorBrush x:Key="BorderBrush"
        Color="#3D3D4F"/>
    <SolidColorBrush x:Key="SidebarBrush"
        Color="#252535"/>
    <SolidColorBrush x:Key="HoverBackgroundBrush"
        Color="#3D3D4F"/>
    <SolidColorBrush x:Key="SelectedItemBrush"
        Color="#1E3A5F"/>
</ResourceDictionary>
```

Переключение в ProfilePopup

```
<!-- ProfilePopup.xaml -->
<ToggleButton IsChecked="{Binding IsDarkMode, Mode=TwoWay}"
    Command="{Binding ToggleThemeCommand}"
    Content="{Binding IsDarkMode,
        Converter={StaticResource ThemeTextConv}}"/>

// ProfileViewModel.cs
private void OnToggleTheme(object? param)
{
    IsDarkMode = !IsDarkMode;
    _themeService.SetTheme(IsDarkMode);
}
```

2.10 Локализация

Приложение поддерживает два языка: русский (ru) и английский (en). Переключение происходит в runtime без перезапуска.

Файлы локализации

ru.xaml — русский

```
<ResourceDictionary>
    <system:String x:Key="AppTitle">Магазин приложений</system:String>
    <system:String x:Key="SearchPlaceholder">Поиск...</system:String>
    <system:String x:Key="CategoryAll">Все</system:String>
    <system:String x:Key="CategoryGames">Игры</system:String>
    <system:String x:Key="CategoryUtilities">Утилиты</system:String>
    <system:String x:Key="CategoryProductivity">
        Продуктивность</system:String>
    <system:String x:Key="CategoryMedia">Медиа</system:String>
    <system:String x:Key="BtnInstall">Установить</system:String>
    <system:String x:Key="BtnUninstall">Удалить</system:String>
    <system:String x:Key="BtnOpen">Открыть</system:String>
    <system:String x:Key="LblDownloads">Скачиваний</system:String>
    <system:String x:Key="LblRating">Рейтинг</system:String>
    <system:String x:Key="LblSize">Размер</system:String>
    <system:String x:Key="LblDeveloper">Разработчик</system:String>
    <system:String x:Key="LblFree">Бесплатно</system:String>
    <system:String x:Key="LblProfile">Профиль</system:String>
    <system:String x:Key="LblLogout">Выйти</system:String>
    <!-- ... -->
</ResourceDictionary>
```

en.xaml — английский

```
<ResourceDictionary>
    <system:String x:Key="AppTitle">App Store</system:String>
    <system:String x:Key="SearchPlaceholder">Search...</system:String>
    <system:String x:Key="CategoryAll">All</system:String>
    <system:String x:Key="CategoryGames">Games</system:String>
    <system:String x:Key="CategoryUtilities">Utilities</system:String>
    <system:String x:Key="CategoryProductivity">
        Productivity</system:String>
    <system:String x:Key="BtnInstall">Install</system:String>
    <system:String x:Key="BtnUninstall">Uninstall</system:String>
    <system:String x:Key="LblFree">Free</system:String>
    <!-- ... -->
</ResourceDictionary>
```

Использование в XAML

```
<!-- DynamicResource для строк локализации -->
<TextBlock Text="{DynamicResource AppTitle}"/>
<TextBlock Text="{DynamicResource SearchPlaceholder}"/>
<Button Content="{DynamicResource BtnInstall}"/>
```

Адаптация PriceConverter

PriceConverter определяет текущий язык из MergedDictionaries[1] и адаптирует:

- Текст «Бесплатно» / «Free» для нулевой цены
- Символ валюты: ₽ для русского, $ для английского

2.11 NuGet-зависимости

Проект использует минимальный набор NuGet-пакетов:

Пакет	Версия	Назначение
Newtonsoft.Json	13.0.3	Сериализация/десериализация JSON (хранение данных)
Microsoft.EntityFrameworkCore	8.0.0	ORM для работы с базами данных
Microsoft.EntityFrameworkCore.Design	8.0.0	Инструменты проектирования EF Core (миграции)
Npgsql.EntityFrameworkCore.PostgreSQL	8.0.0	Провайдер PostgreSQL для EF Core

Файл проекта AppStore.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
    <PropertyGroup>
        <OutputType>WinExe</OutputType>
        <TargetFramework>net8.0-windows</TargetFramework>
        <UseWPF>true</UseWPF>
        <Nullable>enable</Nullable>
        <ImplicitUsings>enable</ImplicitUsings>
    </PropertyGroup>

    <ItemGroup>
        <PackageReference Include="Newtonsoft.Json"
            Version="13.0.3" />
        <PackageReference
            Include="Microsoft.EntityFrameworkCore"
            Version="8.0.0" />
        <PackageReference
            Include="Microsoft.EntityFrameworkCore.Design"
            Version="8.0.0" />
        <PackageReference
            Include="Npgsql.EntityFrameworkCore.PostgreSQL"
            Version="8.0.0" />
    </ItemGroup>

    <ItemGroup>
        <Resource Include="Resources/Cursors/arrow.cur"/>
    </ItemGroup>
</Project>
```

Конец документации

Документ подготовлен автоматически. Версия 1.0, 29 марта 2026.